esprit Se former autrement HONORIS UNITED UNIVERSITIES	Année Universitaire : 2025-2026 **Workshops Framework Symfony6.4** **UP-Web**
Workshop : **Les tests unitaires dans un projet Symfony**	

Introduction

Dans le cadre du projet, le travail s'organise en plusieurs phases : conception, développement (entités, CRUD, services), **phase de test**, puis livraison finale.

La phase de test intervient **après la phase de développement**. Elle a pour objectif de valider et de sécuriser le code réalisé avant la livraison du projet.

Il existe **plusieurs types de tests** dans un projet logiciel (tests unitaires, tests fonctionnels, tests d'intégration). Dans ce workshop, nous nous concentrons sur **les tests unitaires**, car ils constituent la première étape de la phase de test. Ils permettent de vérifier, de manière simple et isolée, la logique interne du code avant d'aborder des tests plus globaux.

⇨ **Les tests unitaires permettent de valider les règles métier, de sécuriser les évolutions et d'améliorer la qualité globale du projet.**

Première approche : création d'un test avec make:test

1. Symfony fournit un outil permettant de générer automatiquement la structure d'un test avec la commande : **php bin/console make:test .** Cette commande propose plusieurs types de tests.

2. Pour les tests unitaires, il faut choisir : **TestCase**

```
C:\xampp2\htdocs\testMe_Symfony5>php bin/console make:test

 Which test type would you like?:
  [TestCase       ] basic PHPUnit tests
  [KernelTestCase ] basic tests that have access to Symfony services
  [WebTestCase    ] to run browser-like scenarios, but that don't execute JavaScript code
  [ApiTestCase    ] to run API-oriented scenarios
  [PantherTestCase] to run e2e scenarios, using a real-browser or HTTP client and a real web server
 >
```

Puis donner un nom à la classe de test, par exemple : **AuthorManagerTest .**

PARTIE PRATIQUE

1. Règles métier à valider :

Pour l'entité Author, on définit les règles suivantes :

1. Le nom de l'auteur est obligatoire
2. L'email doit être valide

⇨ Ces règles doivent être validées par des tests unitaires.

2. Organisation des dossiers

Pour une bonne organisation du projet, on adopte la structure suivante :

```
src/
└── Service/
└── AuthorManager.php

tests/
└── Service/
└── AuthorManagerTest.php
```

Le fichier généré par make:test peut être déplacé dans **tests/Service/**

3. Création du service métier

```php
<?php
namespace App\Service;
use App\Entity\Author;

class AuthorManager
{
   public function validate(Author $author): bool
   {
      if (empty($author->getName())) {
         throw new \InvalidArgumentException('Le nom est obligatoire');
      }

      if (!filter_var($author->getEmail(), FILTER_VALIDATE_EMAIL)) {
         throw new \InvalidArgumentException('Email invalide');}

 return true;
   }
}
```

- ⇨ La validation de l'entité Author repose sur l'utilisation de **empty()** pour vérifier que le nom n'est pas vide, de **FILTER_VALIDATE_EMAIL** pour contrôler le format de l'email, et de **InvalidArgumentException** pour signaler immédiatement toute donnée invalide lors des tests unitaires.

4. Implémentation du test unitaire généré (AuthorManagerTest.php)

```php
<?php

namespace App\Tests\Service;

use App\Entity\Author;
use App\Service\AuthorManager;
use PHPUnit\Framework\TestCase;

class AuthorManagerTest extends TestCase
{
    public function testValidAuthor()
    {
        $author = new Author();
        $author->setName('Victor Hugo');
        $author->setEmail('victor.hugo@gmail.com');

        $manager = new AuthorManager();

        $this->assertTrue($manager->validate($author));
    }

    public function testAuthorWithoutName()
    {
        $this->expectException(\InvalidArgumentException::class);

        $author = new Author();
        $author->setEmail('test@gmail.com');

        $manager = new AuthorManager();
        $manager->validate($author);
    }

    public function testAuthorWithInvalidEmail()
    {
        $this->expectException(\InvalidArgumentException::class);

        $author = new Author();
        $author->setName('Author Test');
        $author->setEmail('email_invalide');

        $manager = new AuthorManager();
        $manager->validate($author);}}
```

- ⇨ Cette classe de test vérifie automatiquement que le service AuthorManager valide un auteur correct et rejette les données invalides (nom vide ou email incorrect) conformément aux règles métier définies.

5. Exécution des tests :

```
php bin/phpunit
```

Résultat attendu :

- ⇨ chaque point correspond à un test
- ⇨ OK indique que la logique métier est valide

Conclusion

Les tests unitaires constituent la **première étape de la phase de test**.
Ils permettent de valider la logique métier et de sécuriser le projet avant la livraison finale.

Travail demandé aux étudiants

Chaque étudiant doit :

1. Choisir une entité de son projet
2. Identifier au moins deux règles métier
3. Créer un service métier correspondant
4. Générer un test avec make:test
5. Implémenter les tests unitaires
6. Vérifier l'exécution des tests

Voici 10 exemples des règles métier :

1. « La quantité saisie ne peut pas être négative. »
2. « Le prix d’un article doit toujours être supérieur à zéro. »
3. « La date de fin d’un événement doit être postérieure à la date de début. »
4. « Un champ obligatoire doit être rempli avant de soumettre le formulaire. »
5. « Une remise ne peut être appliquée que si le montant total dépasse 100. »
6. « Le code saisi doit être unique dans le système. »
7. « Une opération ne peut être validée que si toutes les étapes précédentes sont complétées. »
8. « La durée d’un abonnement ne peut pas être inférieure à un mois. »
9. « Le mot de passe doit contenir au moins 8 caractères. »
10. « Une action critique ne peut être effectuée qu’avec une autorisation préalable. »